

The p -Median Problem: Model and Greedy Heuristic

1 The p -median model

We are given a set of n demand points (hexes) and the same set is used as the set of candidate facility sites (medians). Let

- $V = \{1, \dots, n\}$ be the set of hexes (both demands and candidate sites);
- $d_{ij} \geq 0$ be the distance between hex i and hex j ;
- p be the number of facilities (open sites) to select.

In this implementation the distance is the Euclidean distance between hex grid locations,

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2},$$

where (x_i, y_i) is the location of hex i .

Decision variables

x_{ij} the fraction of demand i served by site j , $i, j \in V$.

There is no separate facility-open variable. The role of y_j is played by the *diagonal* entry x_{jj} : a hex j serves itself exactly when it is open, so x_{jj} is the open indicator and is the only variable that must be integral,

$$x_{jj} \in \{0, 1\} \quad j \in V, \quad 0 \leq x_{ij} \leq 1 \quad (i \neq j).$$

The off-diagonal assignments x_{ij} ($i \neq j$) stay continuous in $[0, 1]$. Once the p open sites are fixed by the binary diagonal, the remaining assignment problem is totally unimodular, so the continuous x_{ij} take values in $\{0, 1\}$ at the optimum anyway.

Formulation

$$\min_x \quad \sum_{i \in V} \sum_{j \in V} d_{ij} x_{ij} \tag{1}$$

$$\text{s.t.} \quad \sum_{j \in V} x_{ij} = 1 \quad \forall i \in V, \tag{2}$$

$$x_{ij} \leq x_{jj} \quad \forall i, j \in V, \tag{3}$$

$$\sum_{j \in V} x_{jj} = p, \tag{4}$$

$$x_{jj} \in \{0, 1\} \quad \forall j \in V, \tag{5}$$

$$0 \leq x_{ij} \leq 1 \quad \forall i \neq j. \tag{6}$$

The objective (??) minimises the total assignment distance. Constraint (??) forces every hex to be fully assigned, (??) allows a hex to be served by site j only if j is open (i.e. $x_{jj} = 1$), and (??) opens exactly p facilities.

Weighted demand

Each hex carries a demand weight $w_h \geq 0$ (in the code, `Hex::weight`, default 1). The weighted p -median replaces the objective (??) with

$$\min_x \sum_{i \in V} \sum_{j \in V} w_i d_{ij} x_{ij},$$

so that hexes with larger demand pull facilities closer. The weights also drive the workload-balancing repair pass below, where the *load* of a district is the sum of the weights of the hexes assigned to it.

2 Greedy heuristic

The greedy heuristic builds the solution by repeatedly opening the candidate site that reduces the total assignment distance the most. Let c_h be the distance from hex h to its nearest currently open site; opening site c would change the total cost to $\sum_{h \in V} \min(c_h, d_{hc})$.

Algorithm 1 Greedy p -median (site selection)

Require: hexes V , distances d , facility count p

Ensure: set of open sites S

```

1:  $c_h \leftarrow \infty$  for all  $h \in V$                                  $\triangleright$  cost of each hex to its nearest open site
2:  $S \leftarrow \emptyset$ 
3: for  $k \leftarrow 1$  to  $p$  do
4:    $c^* \leftarrow \arg \min_{c \in V \setminus S} \sum_{h \in V} \min(c_h, d_{hc})$      $\triangleright$  site giving the lowest total cost
5:    $S \leftarrow S \cup \{c^*\}$ 
6:   for  $h \in V$  do
7:      $c_h \leftarrow \min(c_h, d_{hc^*})$                                  $\triangleright$  update nearest-site costs
8:   end for
9: end for
10: return  $S$ 

```

Once the p sites are chosen, each hex is assigned to its nearest open site:

$$\text{assign}(h) = \arg \min_{s \in S} d_{hs}, \quad h \in V,$$

and the objective is $\sum_{h \in V} d_{h, \text{assign}(h)}$.

The greedy solution may then be passed through optional repair passes (contiguity and workload balancing) before reporting the final objective.

3 Repair passes

The repair passes operate in *district-index space*: $a_h \in \{1, \dots, p\}$ is the index of the open site currently serving hex h . They rely on the hex adjacency graph $N(h)$, the set of grid neighbours of h (the eight king-move offsets on the hex grid).

Algorithm 2 Nearest-site assignment

Require: hexes V , distances d , open sites S **Ensure:** assignment a

```
1: for  $h \in V$  do  
2:    $a_h \leftarrow \arg \min_{s \in S} d_{hs}$   
3: end for  
4: return  $a$ 
```

Contiguity repair

A district should be a single connected region. For each district s , a breadth-first search from its open site S_s marks every hex of s reachable through neighbours that also belong to s . Any hex of s that is *not* reachable is “stranded” and is reassigned to a neighbouring district.

Algorithm 3 Contiguity repair

Require: assignment a , adjacency N , open sites S **Ensure:** repaired assignment a

```
1: for  $s \leftarrow 1$  to  $p$  do  
2:    $R \leftarrow \text{BFS}(S_s, N, a, s)$  ▷ hexes of  $s$  reachable from its site  
3:   for  $h \in V$  with  $a_h = s$  and  $h \notin R$  do  
4:     if  $\exists j \in N(h)$  with  $a_j \neq s$  then  
5:        $a_h \leftarrow a_j$  ▷ move stranded hex to a neighbour's district  
6:     end if  
7:   end for  
8: end for  
9: return  $a$ 
```

Here $\text{BFS}(\text{start}, N, a, s)$ returns the set of hexes reachable from start using only edges of N whose endpoints satisfy $a_j = s$.

Workload-balancing repair

This pass evens out district loads until the most loaded district is within a factor **limit** of the mean load. Let $\text{load}(s) = \sum_{h: a_h = s} w_h$ and $\overline{\text{load}} = \frac{1}{p} \sum_s \text{load}(s)$. While the imbalance $\max_s \text{load}(s) / \overline{\text{load}}$ exceeds the limit, a single boundary hex is moved from the most loaded district to the least loaded one, provided the move keeps the donor district connected.

$\text{CONNECTEDWITHOUT}(a, N, S_o, h, o)$ tests, via a BFS from the site S_o that skips hex h , whether district o would remain connected after h is removed (i.e. the BFS reaches all but one of the district's hexes). The loop also stops if no admissible move exists or after *maxIter* iterations, guaranteeing termination.

Algorithm 4 Workload-balancing repair

Require: assignment a , weights w , adjacency N , open sites S , ratio $limit$

Ensure: repaired assignment a

```
1: for  $iter \leftarrow 1$  to  $maxIter$  do
2:   compute  $load(s)$  for all  $s$ , and mean  $\overline{load}$ 
3:   if  $\overline{load} = 0$  or  $\max_s load(s)/\overline{load} \leq limit$  then
4:     break
5:   end if
6:    $o \leftarrow \arg \max_s load(s)$ ;  $u \leftarrow \arg \min_s load(s)$ 
7:   if  $o = u$  then
8:     break
9:   end if
10:   $swapped \leftarrow \text{false}$ 
11:  for  $h \in V$  with  $a_h = o$  and  $h \neq S_o$  do
12:    if  $\exists j \in N(h)$  with  $a_j = u$  and  $CONNECTEDWITHOUT(a, N, S_o, h, o)$  then
13:       $a_h \leftarrow u$ ;  $swapped \leftarrow \text{true}$ ; break
14:    end if
15:  end for
16:  if not  $swapped$  then
17:    break
18:  end if
19: end for
20: return  $a$ 
```
